Automated Detection of Pork Rasher Packaging Errors

MMA3001 - Numerical Methods and Machine Learning | Quoc Hung Vo | Monash University

Abstract

This project tests an image-based method for detecting visible defects in pork rasher packaging. A pretrained YOLOv8n detector was fine-tuned on a cleaned version of the Roboflow Version 4 dataset. I also implemented spatial finite differences from MMA3001 Week 7 to study image gradients and check how blur affects image information. Numerical verification, controlled validation experiments and one withheld test evaluation were completed. The final test mAP50 was 0.4609, so the detector is useful for investigation but not yet reliable enough for autonomous production inspection [1, 2].

1. Engineering problem and scope

In food packaging, defects such as loose meat or an unsealed tray may need further inspection before a product is distributed. The engineering task is to locate visible defects from an RGB image so an inspector can review the affected area. The five dataset classes are loose-meat, packaging-error, twisted-meat, unsealed and wrinkle [2, 3].

The project outputs candidate defect locations, not a certified pass/fail decision. It cannot measure sealing force, leakage or food safety from an image alone. The method was developed and tested on the supplied image dataset, not on a working production line. Many images in the dataset show only part of a pork rasher tray. Therefore, the detector can only identify defects within the visible area. A prediction with no detected defect does not necessarily mean the entire package is defect-free. A complete inspection system would require sufficient camera coverage of the whole tray.

1.1 Main inputs and outputs

Table 1. Input/output definition and units.

Item	Format and meaning
Image	RGB, 8-bit per channel (0-255); supplied images described as 720 x 540 pixels.
Labels	YOLO .txt files: class ID 0-4 plus box centre, width and height normalised to [0,1].
Detector output	Predicted boxes in image pixels, class IDs and confidence values [0,1].
Numerical output	Grayscale gradient magnitude in normalised intensity per pixel (pixel spacing = 1).
Evaluation output	Precision, recall, mAP50, mAP50-95 (unitless) and forward-pass ms/image.

Malformed or missing label pairs are detected during audit. Empty annotations are kept as possible negative samples. The image-analysis code checks invalid arrays and unsupported or unreadable image files.



Figure 1. **Example image from the Pork Rasher Error-Packaging Version 4 dataset [3].**

2. Dataset preparation and model development

This project used Dataset 3 provided for MMA3001, based on the Pork Rasher Error-Packaging Version 4 dataset available on Roboflow Universe [2, 3]. The dataset README identifies it as user-provided and licensed under CC BY 4.0. It provided 3,549 images in predefined splits. An audit found 13 filename source IDs shared across splits. The cleaning rule gave priority to test, then validation, then training. Images and their labels were removed together, leaving 3,512 images. The original ZIP was unchanged.

Table 2. Dataset audit and cleaning results.

Split	Original images	Removed	Cleaned images
Training	3,349	36	3,313
Validation	120	1	119
Test	80	0	80
Total	3,549	37	3,512

This cleaning removed the detected source-ID overlaps, but similar video frames could still remain in different splits. Empty annotation files were not automatically treated as incorrect. The training images are augmented in the supplied dataset, while the validation and test sets were kept separate for evaluation [2, 3].

2.1 YOLOv8n detector

I fine-tuned one pretrained Ultralytics YOLOv8n model instead of training a detector from scratch. YOLO predicts bounding boxes, classes and confidence scores, which match the required output [5]. The model processes each RGB image through a neural network to identify visual features associated with packaging defects. Candidate detections are then filtered using non-maximum suppression (NMS) to reduce overlapping predictions. The pretrained model was fine-tuned using annotated training images, while validation data was used to assess performance and select the final configuration.

Table 3. Fixed model training and development settings.

Setting	Recorded value
Training data	3,313 cleaned images; 5 defect classes
Model and input	Pretrained YOLOv8n; image size 640
Training configuration	25 epochs completed; batch size 4; patience 7; seed 42
Machine	Windows 11, Python 3.12.10, RTX 4050 Laptop GPU (CUDA)
Training runtime	0.473 hours (about 28.4 minutes)
Fixed follow-up validation	Precision 0.5936; recall 0.4977; mAP50 0.5551; mAP50-95 0.3310

The saved best.pt checkpoint is included in the public GitHub repository. Phase 1 audit and cleaning checks passed 40/40 pytest tests. These tests support software correctness, but they do not prove that the detector recognises defects accurately. The test split was held back until the final evaluation.

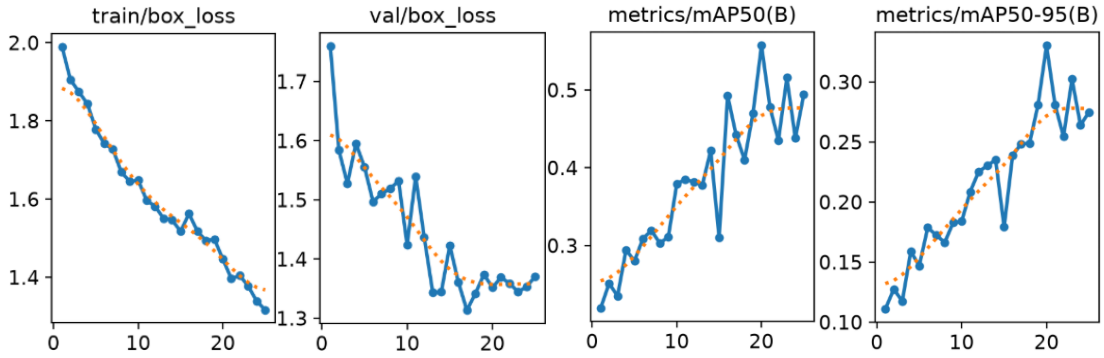


Figure 2. **YOLOv8n training box loss, validation box loss, mAP50 and mAP50–95 over 25 epochs.**

3. Numerical method: finite differences

Finite differences estimate derivatives using values at neighbouring grid points. This method was taught in MMA3001 Week 7, Section 7.1 [4]. In this project, a grayscale image is treated as a two-dimensional intensity field, I(x,y), normalised to [0,1].

For interior pixels, second-order centred differences are used to approximate the horizontal and vertical derivatives:

$$D_x I_{i,j} \approx \frac{I_{i,j+1} - I_{i,j-1}}{2h} \quad D_y I_{i,j} \approx \frac{I_{i+1,j} - I_{i-1,j}}{2h}$$

At image boundaries, neighbouring values outside the image are unavailable. Therefore, first-order forward or backward differences are used instead.

$$D_{\text{forward}} I \approx \frac{I_1 - I_0}{h} \quad D_{\text{backward}} I \approx \frac{I_n - I_{n-1}}{h}$$

The horizontal and vertical derivatives are combined to calculate the gradient magnitude:

$$G_{i,j} = \sqrt{(D_x I_{i,j})^2 + (D_y I_{i,j})^2}$$

For image analysis, the pixel spacing is h = 1 pixel. The resulting gradient magnitude represents image edge strength in normalised intensity per pixel, rather than a physical measurement in millimetres [4].

3.1 Verification against known functions

I tested the NumPy implementation using constant and linear fields, boundary cases, invalid inputs and the smooth function f(x,y) = sin(x) + cos(y). Its exact derivatives are cos(x) and -sin(y). By reducing grid spacing h, I could check whether the numerical error decreased at the expected rate [4].

Table 4. Numerical verification results from finite_difference_verification.txt.

Check	Observed result
All synthetic checks	10/10 PASS
Interior centred differences	Observed convergence order near 2 (expected O(h^2))
One-sided boundary differences	Observed convergence order near 1 (expected O(h))
Finest-grid max error	Interior 9.954 x 10^-6; boundary 3.826 x 10^-3
Invalid inputs	7/7 tested invalid cases rejected

The larger boundary error is expected because the one-sided stencil is first-order accurate. Finite differences also amplify noise, especially for small h. Passing these tests shows that the derivative formulas were implemented correctly on known fields; it does not confirm the quality of YOLO predictions.

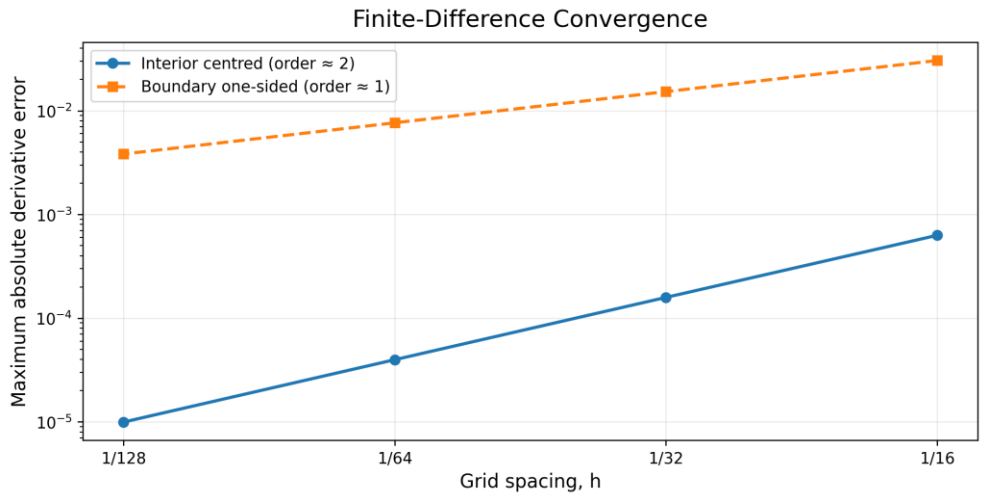


Figure 3. **Finite-difference convergence verification showing second-order accuracy for interior centred differences and first-order accuracy at image boundaries.**

4. Blur sensitivity and inference optimisation

4.1 Gaussian blur experiment

I blurred the same 119 validation images at sigma = 0, 1 and 2 pixels, keeping the annotations and trained model fixed. The mean gradient magnitude fell as blur increased. YOLO validation mAP50 also decreased, but not in the same proportion.

Table 5. Measured blur sensitivity on 119 validation images.

Sigma (px)	Mean gradient	Gradient change	YOLO mAP50	mAP50 change
0	0.032140	0%	0.5551	0%
1	0.024323	-24.319%	0.4874	-12.204%
2	0.016088	-49.944%	0.4830	-12.997%

The result shows that this detector is sensitive to image blur. However, a lower gradient magnitude is not the same as a proportional loss of detection accuracy. At sigma = 2, precision rose while recall fell. This is one controlled experiment, not evidence of a general relationship between gradients and detection performance.

4.2 Optimising inference input size

The same trained checkpoint was evaluated at input sizes 480, 640 and 800 on validation data. Batch size, dataset and GPU were kept fixed, so input size was the main variable. The comparison was used for model selection before checking the final test.

Table 6. Accuracy-speed trade-off for input size.

Input size	mAP50	mAP50-95	Inference (ms/image)
480	0.5296	0.2804	2.0788
640 (selected)	0.5551	0.3310	2.9033
800	0.3988	0.2289	3.9283

The 480-pixel configuration reduced inference time by 28.4% relative to 640 pixels, but its mAP50-95 was 15.3% lower. The 800-pixel configuration was both slower and less accurate. Therefore, I retained the 640-pixel setting because detection accuracy was prioritised over inference speed. This decision was based on validation results before the withheld test evaluation. Reported timings are from single runs and do not represent complete camera-to-inspection latency.

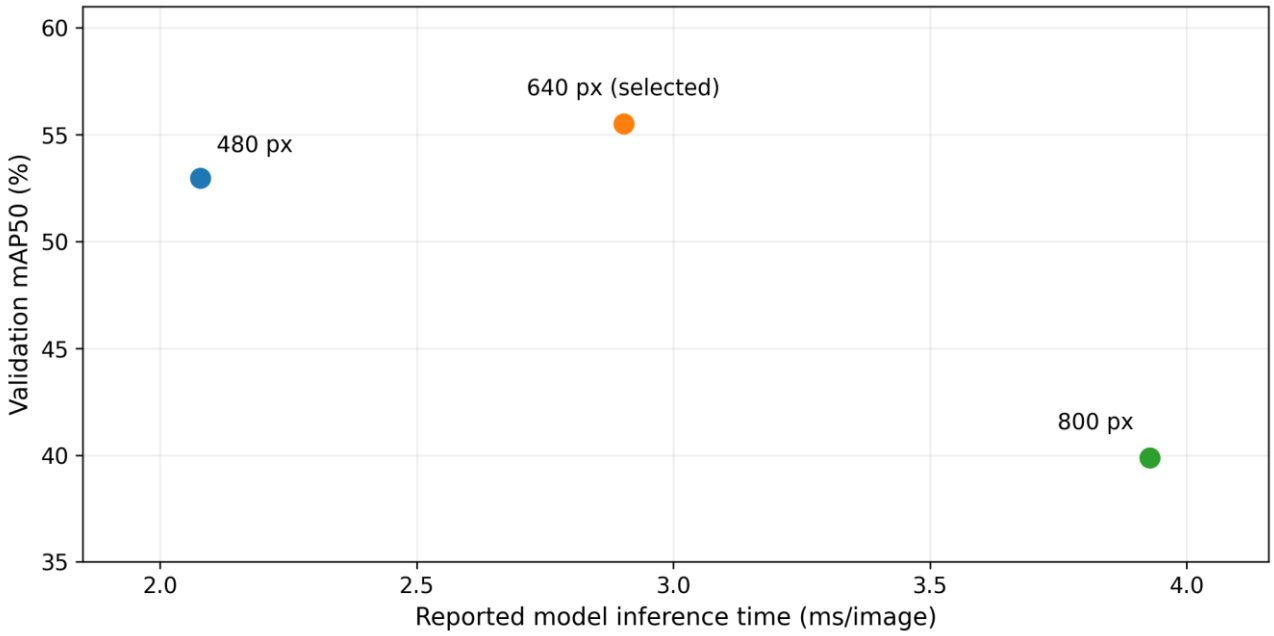


Figure 4. **Validation mAP50 and inference time for three input resolutions (480, 640 and 800 pixels).**

5. Final evaluation and engineering interpretation

After selecting the checkpoint and 640-pixel input size, I ran the final evaluation on 80 withheld test images with 79 annotated boxes. No test-informed retraining or further parameter selection was done. The evaluation used batch 4, CUDA, conf = 0.001 for candidate filtering and NMS IoU = 0.7. The confidence setting is not a production rejection threshold.

Table 7. Overall fixed-condition validation and test results.

Metric	Validation (119 images)	Test (80 images)
Precision	0.5936	0.3364
Recall	0.4977	0.4766
mAP50	0.5551	0.4609
mAP50-95	0.3310	0.2365
F1 (aggregate P and R)	-	0.3944

Precision measures the share of predicted objects that match labels, while recall measures the share of labelled objects detected. mAP50 uses 0.50 box overlap; mAP50-95 averages across stricter overlap thresholds [6]. The test performance was lower than validation, so the validation score alone would overstate confidence in the detector.

Table 8. Final test results by defect class; N/A means not evaluable.

Class	GT boxes	Precision	Recall	mAP50
loose-meat	3	0.5712	0.3333	0.4689
packaging-error	23	0.2686	0.6957	0.5073
twisted-meat	4	0.0000	0.0000	0.0153
unsealed	49	0.5058	0.8776	0.8523
wrinkle	0	N/A	N/A	N/A

Unsealed was the strongest measured class (recall 0.8776), but its precision was only 0.5058. Packaging-error precision was 0.2686, so false detections would be a concern in an inspection system. Twisted-meat had zero precision and recall at the reported operating point, and only four labelled boxes. Wrinkle had no test boxes and therefore cannot be evaluated, rather than having zero accuracy. The differences between classes may partly reflect the uneven number of annotated examples. For example, unsealed had 49 test annotations, while twisted-meat had only four. However, class frequency alone cannot explain the observed performance. Defect appearance, image quality and annotation consistency may also affect detection, and further experiments would be needed to separate these factors.



***Figure 5*. Ground-truth annotation (a) and YOLOv8n prediction (b) for test_0001 from the withheld test set. The packaging-error annotation was not detected by the model, illustrating a false negative.**

6. Discussion, limitations and conclusion

6.1 Alternative method and main limitations

A rule-based approach using thresholding and edge detection would be simpler and more interpretable than YOLOv8n. However, meat boundaries, plastic reflections and tray edges can resemble defects, requiring manual rules to distinguish the five classes. YOLOv8n was selected to learn class-specific features and locate defects. The alternative was not implemented, so no performance comparison is claimed. Instead, three YOLO input resolutions were evaluated.

The test set was imbalanced, with 49 unsealed, three loose-meat, four twisted-meat and no wrinkle annotations. Filename-based split cleaning could not exclude visually similar neighbouring frames. Synthetic blur does not represent all real camera conditions, and GPU inference times exclude image acquisition. Therefore, the model is not ready for automatic product rejection.

6.2 Reproducibility and verification

The GitHub repository includes seven Python modules, the trained best.pt file, a README, dependencies, pdoc HTML documentation, 40 pytest results and the CSV evidence used in this report. The original dataset must be obtained from Roboflow [3]. A cloned repository was checked using the existing Python environment, but a fresh installation on another machine was not tested. The model and numerical calculations were evaluated separately: software verification shows that the code runs as intended, while labelled images are needed to judge whether it solves the inspection problem.

6.3 AI use and reflection

I used ChatGPT for project planning, understanding numerical methods, interpreting results, and drafting and revising this report. Codex generated most of the initial Python code and assisted with debugging. AI made a substantial contribution to code development and report preparation, while I selected the dataset, defined the scope, ran experiments, checked results and chose the final configuration.

I verified AI-assisted work through dataset auditing, 40 automated tests, 10 numerical checks and recorded evaluation results. One generated error, validator.nt_per_class, caused an AttributeError and was corrected to validator.metrics.nt_per_class. This showed me the importance of verifying generated code rather than accepting it automatically. AI improved efficiency, but understanding the results and identifying limitations still required my own judgement.

6.4 Conclusion

This project produced a working defect-localisation workflow and a finite-difference method that passed 10/10 numerical checks. Known split overlaps were reduced by removing 37 images. Blur reduced measured edge strength and validation mAP50, while 640 pixels gave the best validation accuracy in the resolution comparison. The final test reached mAP50 0.4609 and mAP50-95 0.2365. The system is a useful prototype for studying image inspection, but rare-class evidence and overall detection performance are not sufficient for real production use. Future work should include more rare-defect examples, independent images from new packaging runs and full end-to-end inspection timing. A wider camera view or multiple coordinated images would also be necessary to assess the entire tray rather than only its visible regions.

References

[1] Monash University, MMA3001 Numerical Methods and Machine Learning: Project Brief, 2026, pp. 2, 4-8, 10-15. Unit-provided document.

[2] Monash University, MMA3001 Project Datasets, 2026, Dataset 3: Automated Detection of Pork Rasher Packaging Errors and Meat Quality, p. 3. Unit-provided document.

[3] Roboflow Universe, “Pork Rasher Error-Packaging,” Dataset Version 4, user-provided dataset, licensed under CC BY 4.0. Available: https://universe.roboflow.com/hello-2aqe0/pork-rasher-error-packaging/dataset/4.

[4] K. Granland, MMA3001 Week 7: Finite Differences and Ordinary Differential Equations, Section 7.1, Monash University, 2026, slides 6-9. See also MMA3001_Notes_Week07_1_Finite_Difference_Derivatives.ipynb (unit material).

[5] Ultralytics, “Explore Ultralytics YOLOv8,” official documentation. Available: https://docs.ultralytics.com/models/yolov8/.

[6] Ultralytics, “YOLO Performance Metrics,” official documentation. Available: https://docs.ultralytics.com/guides/yolo-performance-metrics/.

Source data used for the results

Project figures and results refer to the repository files under reports/evidence/: finite_difference_verification.txt, finite_difference_summary.csv, blur_robustness.csv, resolution_comparison.csv, test_overall.csv and test_per_class.csv. These are the recorded experiment outputs, not additional literature references.

Project repository: https://github.com/qvoo0003-eng/mma3001-pork-packaging-inspection